

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 8

Platform-Specific Features

expect/actual, Dependency Injection, dan Platform APIs

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026

OUTLINE PERTEMUAN

01

Review Minggu Lalu

Local Data Storage

02

expect/actual Pattern

Platform-specific implementations

03

Dependency Injection

Koin untuk Kotlin Multiplatform

04

Platform APIs

Akses fitur native (Device Info, Battery, dll)

05

Permissions

Mengelola izin aplikasi

06

Hands-on Practice

Implementasi fitur platform-specific

CAPAIAN PEMBELAJARAN

CPMK0502 dan CPMK0503

Mahasiswa mampu menjelaskan dan menerapkan teknologi sistem cerdas dalam pengembangan perangkat lunak.

Indikator: Mampu mengimplementasikan expect/actual dan menggunakan Dependency Injection

Setelah pertemuan ini, mahasiswa mampu:

- ✓ Memahami dan menerapkan expect/actual pattern
- ✓ Menggunakan Koin untuk Dependency Injection
- ✓ Mengakses platform-specific APIs
- ✓ Mengelola permissions di multiplatform
- ✓ Mengintegrasikan fitur native ke shared code

REVIEW MINGGU LALU

Pertemuan 7: Local Data Storage

DataStore	Key-value storage untuk preferences
------------------	-------------------------------------

SQLDelight	Type-safe SQL database
-------------------	------------------------

Repository	Abstraksi data layer
-------------------	----------------------

Offline-First	Cache-first strategy
----------------------	----------------------

expect/actual	DatabaseDriverFactory per platform
----------------------	------------------------------------

🕒 Hari ini: Mendalami expect/actual dan mengakses lebih banyak platform APIs

BAGIAN 1

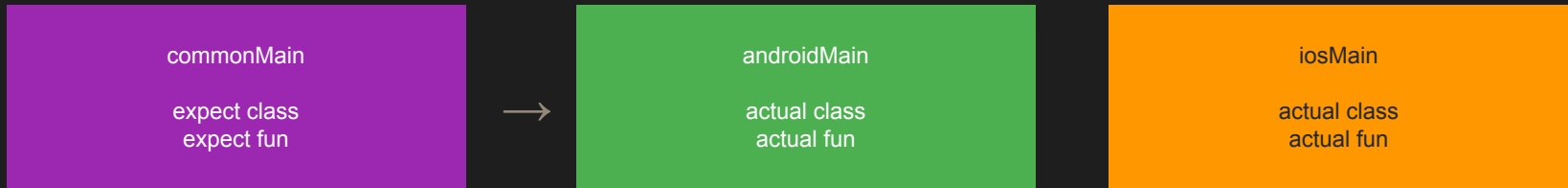
EXPECT/ACTUAL PATTERN

Platform-specific implementations in KMP

APA ITU EXPECT/ACTUAL?

expect/actual adalah mekanisme di Kotlin Multiplatform untuk menulis code yang berbeda di setiap platform sambil mempertahankan API yang sama di common code.

Cara Kerja:



- expect = Deklarasi API di common code (tanpa implementasi)
- actual = Implementasi spesifik di setiap platform
- Compiler memastikan semua expect memiliki actual

CONTOH: PLATFORM NAME

commonMain

```
// Platform.kt
expect fun getPlatformName(): String

// Bisa digunakan di common code
fun greet(): String = "Hello from ${getPlatformName()}!"
```

androidMain

```
// Platform.android.kt
actual fun getPlatformName(): String {
    return "Android ${Build.VERSION.SDK_INT}"
}
```

iosMain

```
// Platform.ios.kt
actual fun getPlatformName(): String {
    return UIDevice.currentDevice.systemName()
}
```



Output:

```
Android: "Hello from Android 34!"
iOS: "Hello from iOS!"
```

EXPECT CLASS

DeviceInfo.kt

```
// commonMain - DeviceInfo.kt
expect class DeviceInfo() {
    fun getDeviceName(): String
    fun getOsVersion(): String
    fun isTablet(): Boolean
}

// androidMain - DeviceInfo.android.kt
actual class DeviceInfo {
    actual fun getDeviceName(): String = Build.MODEL
    actual fun getOsVersion(): String = "Android ${Build.VERSION.RELEASE}"
    actual fun isTablet(): Boolean {
        val metrics = Resources.getSystem().displayMetrics
        val diag = sqrt((metrics.widthPixels/metrics.xdpi).pow(2) +
            (metrics.heightPixels/metrics.ydpi).pow(2))
        return diag >= 7.0
    }
}

// iosMain - DeviceInfo.ios.kt
actual class DeviceInfo {
    actual fun getDeviceName(): String = UIDevice.currentDevice.name
    actual fun getOsVersion(): String = "iOS ${UIDevice.currentDevice.systemVersion}"
    actual fun isTablet(): Boolean =
        UIDevice.currentDevice.userInterfaceIdiom == UIUserInterfaceIdiomPad
}
```

KAPAN MENGGUNAKAN EXPECT/ACTUAL?



Database Driver

SQLDelight driver berbeda per platform



Device Info

Model, OS version, screen size



Secure Storage

Keychain (iOS) vs Keystore (Android)



Location Services

GPS APIs berbeda per platform



Camera/Media

Akses kamera dan galeri



Notifications

Push notification setup

BAGIAN 2

DEPENDENCY INJECTION

Koin untuk Kotlin Multiplatform

APA ITU DEPENDENCY INJECTION?

Dependency Injection (DI) adalah design pattern dimana objek menerima dependencies-nya dari luar, bukan membuatnya sendiri. Ini membuat code lebih testable, modular, dan maintainable.

Tanpa DI vs Dengan DI:

✗ Tanpa DI

```
class NotesViewModel {  
    // Tightly coupled!  
    private val repository =  
        NoteRepository(  
            AppDatabase(  
                DatabaseDriverFactory(ctx)  
            )  
        )  
    // Hard to test  
}
```

✓ Dengan DI

```
class NotesViewModel(  
    // Injected from outside  
    private val repository:  
        NoteRepository  
) {  
    // Loosely coupled!  
    // Easy to test with mock  
    // Easy to swap impl  
}
```

Keuntungan DI: Testability, Modularity, Flexibility, Separation of Concerns

KOIN: DI FRAMEWORK UNTUK KMP

Koin adalah lightweight dependency injection framework untuk Kotlin. Pragmatic, tidak ada code generation, dan mendukung Kotlin Multiplatform!

✓ Pure Kotlin DSL

Deklarasi dependencies dengan Kotlin syntax

✓ No Code Generation

Tidak seperti Dagger/Hilt

✓ Multiplatform Ready

Works on Android, iOS, Desktop, Web

✓ Lightweight

Minimal overhead, fast startup

✓ Easy Testing

Built-in support untuk mocking

SETUP KOIN

build.gradle.kts

```
kotlin {
    sourceSets {
        commonMain.dependencies {
            // Koin Core
            implementation("io.insert-koin:koin-core:3.5.3")

            // Koin untuk Compose (optional)
            implementation("io.insert-koin:koin-compose:1.1.2")
        }

        androidMain.dependencies {
            // Koin Android
            implementation("io.insert-koin:koin-android:3.5.3")
            implementation("io.insert-koin:koin-androidx-compose:3.5.3")
        }
    }
}
```

KOIN MODULE

di/AppModule.kt

```
import org.koin.core.module.dsl.singleOf
import org.koin.core.module.dsl.viewModelOf
import org.koin.dsl.module

// Common module - shared across platforms
val commonModule = module {
    // Singleton: Satu instance untuk seluruh app
    single { AppDatabase(get()) }

    // Factory: Instance baru setiap dipanggil
    factory { NoteRepository(get()) }

    // ViewModel
    viewModelOf(::NotesViewModel)
    viewModelOf(::SettingsViewModel)
}

// Platform-specific module
val platformModule = module {
    single { DatabaseDriverFactory() }
    single { DeviceInfo() }
}

// Combine modules
val appModules = listOf(commonModule, platformModule)
```

INISIALISASI KOIN

Android

```
// Application.kt
class MyApp : Application() {
    override fun onCreate() {
        super.onCreate()
        startKoin {
            androidContext(this@MyApp)
            modules(appModules)
        }
    }
}
```

iOS

```
// KoinHelper.kt (commonMain)
fun initKoin() {
    startKoin {
        modules(appModules)
    }
}

// iOS AppDelegate.swift
KoinHelperKt.initKoin()
```

Compose Multiplatform

```
// App.kt
@Composable
fun App() {
    KoinApplication(application = { modules(appModules) }) {
        MaterialTheme {
            MainScreen()
        }
    }
}
```

MENGGUNAKAN KOIN

Inject Dependencies

```
// Di ViewModel - Constructor Injection
class NotesViewModel(
    private val repository: NoteRepository,
    private val deviceInfo: DeviceInfo
) : ViewModel() {
    // dependencies sudah di-inject
}

// Di Composable - koinViewModel()
@Composable
fun NotesScreen() {
    val viewModel: NotesViewModel = koinViewModel()
    // ...
}

// Di Composable - koinInject()
@Composable
fun SettingsScreen() {
    val deviceInfo: DeviceInfo = koinInject()
    Text("Device: ${deviceInfo.getDeviceName()}")
}

// Lazy injection di class biasa
val repository: NoteRepository by inject()
```

BAGIAN 3

PLATFORM APIs

Mengakses fitur native platform

PLATFORM APIs YANG UMUM

Device Info

Android: `Build.MODEL`, `Build.VERSION`

iOS: `UIDevice.current`

Battery

Android: `BatteryManager`

iOS: `UIDevice.batteryLevel`

Location

Android: `FusedLocationProvider`

iOS: `CLLocationManager`

Camera

Android: `CameraX`

iOS: `AVCaptureSession`

Notifications

Android: `NotificationManager`

iOS: `UNUserNotificationCenter`

Network

Android: `ConnectivityManager`

iOS: `NWPathMonitor`

CONTOH: NETWORK MONITOR

NetworkMonitor.kt

```
// commonMain - NetworkMonitor.kt
expect class NetworkMonitor {
    fun isConnected(): Boolean
    fun observeConnectivity(): Flow<Boolean>
}

// androidMain - NetworkMonitor.android.kt
actual class NetworkMonitor(private val context: Context) {
    private val cm = context.getSystemService(CONNECTIVITY_SERVICE) as ConnectivityManager

    actual fun isConnected(): Boolean {
        val network = cm.activeNetwork ?: return false
        val caps = cm.getNetworkCapabilities(network) ?: return false
        return caps.hasCapability(NET_CAPABILITY_INTERNET)
    }

    actual fun observeConnectivity(): Flow<Boolean> = callbackFlow {
        val callback = object : ConnectivityManager.NetworkCallback() {
            override fun onAvailable(network: Network) { trySend(true) }
            override fun onLost(network: Network) { trySend(false) }
        }
        cm.registerDefaultNetworkCallback(callback)
        awaitClose { cm.unregisterNetworkCallback(callback) }
    }
}
```

CONTOH: BATTERY STATUS

BatteryInfo.kt

```
// commonMain - BatteryInfo.kt
expect class BatteryInfo {
    fun getBatteryLevel(): Int          // 0-100
    fun isCharging(): Boolean
}

// androidMain - BatteryInfo.android.kt
actual class BatteryInfo(private val context: Context) {
    private val batteryManager =
        context.getSystemService(BATTERY_SERVICE) as BatteryManager

    actual fun getBatteryLevel(): Int {
        return batteryManager.getIntProperty(BATTERY_PROPERTY_CAPACITY)
    }

    actual fun isCharging(): Boolean {
        val intent = context.registerReceiver(null,
            IntentFilter(Intent.ACTION_BATTERY_CHANGED))
        val status = intent?.getIntExtra(EXTRA_STATUS, -1) ?: -1
        return status == BATTERY_STATUS_CHARGING
    }
}

// iosMain - BatteryInfo.ios.kt
actual class BatteryInfo {
    init { UIDevice.currentDevice.batteryMonitoringEnabled = true }
    actual fun getBatteryLevel() = (UIDevice.currentDevice.batteryLevel * 100).toInt()
    actual fun isCharging() =
        UIDevice.currentDevice.batteryState == UIDeviceBatteryStateCharging
}
```

BAGIAN 4

PERMISSIONS

Mengelola izin aplikasi

JENIS-JENIS PERMISSION

Normal Permissions

Granted automatically at install

- INTERNET
- ACCESS_NETWORK_STATE
- BLUETOOTH
- VIBRATE
- WAKE_LOCK

Dangerous Permissions

Requires runtime request

- CAMERA
- LOCATION
- CONTACTS
- STORAGE
- MICROPHONE

Permission Flow:

1. Check if permission granted →
2. If not, show rationale (optional)
3. Request permission →
4. Handle result (granted/denied)
5. Handle "Don't ask again" case → Guide user to Settings

PERMISSION HANDLER (KMP)

commonMain

```
// Permission.kt (commonMain)
enum class Permission { CAMERA, LOCATION, STORAGE, NOTIFICATIONS }

enum class PermissionStatus { GRANTED, DENIED, NOT_DETERMINED }

expect class PermissionHandler {
    fun checkPermission(permission: Permission): PermissionStatus
    suspend fun requestPermission(permission: Permission): PermissionStatus
}

// Usage in ViewModel
class CameraViewModel(private val permissionHandler: PermissionHandler) : ViewModel() {

    suspend fun openCamera() {
        when (permissionHandler.checkPermission(Permission.CAMERA)) {
            PermissionStatus.GRANTED -> launchCamera()
            PermissionStatus.DENIED -> showPermissionDeniedDialog()
            PermissionStatus.NOT_DETERMINED -> {
                val result = permissionHandler.requestPermission(Permission.CAMERA)
                if (result == PermissionStatus.GRANTED) launchCamera()
            }
        }
    }
}
```

PERMISSION DI COMPOSE (Accompanist)

Accompanist Permissions

```
// build.gradle.kts
implementation("com.google.accompanist:accompanist-permissions:0.34.0")

// CameraScreen.kt
@OptIn(ExperimentalPermissionsApi::class)
@Composable
fun CameraScreen() {
    val cameraPermissionState = rememberPermissionState(Manifest.permission.CAMERA)

    when {
        cameraPermissionState.status.isGranted -> {
            // Permission granted - show camera
            CameraPreview()
        }
        cameraPermissionState.status.shouldShowRationale -> {
            // Explain why we need permission
            Column {
                Text("Camera permission is needed to take photos")
                Button(onClick = { cameraPermissionState.launchPermissionRequest() }) {
                    Text("Grant Permission")
                }
            }
        }
        else -> {
            // Request permission
            Button(onClick = { cameraPermissionState.launchPermissionRequest() }) {
                Text("Enable Camera")
            }
        }
    }
}
```

BAGIAN 5

HANDS-ON PRACTICE

3 Latihan Praktik

LATIHAN 1: DEVICE INFO

Implementasi expect/actual untuk device info

Template

```
// commonMain
expect class DeviceInfo() {
    fun getDeviceName(): String
    fun getOsVersion(): String
    fun getAppVersion(): String
}

// TODO: Implement actual classes
// androidMain: Build.MODEL, Build.VERSION
// iosMain: UIDevice.current

// Display in Composable
@Composable
fun DeviceInfoScreen() {
    val deviceInfo: DeviceInfo = koinInject()
    Column {
        Text("Device: ${deviceInfo.getDeviceName()}")
        Text("OS: ${deviceInfo.getOsVersion()}")
    }
}
```

Checklist:

- ☐ Create expect class
- ☐ Android actual
- ☐ iOS actual (stub ok)
- ☐ Register di Koin
- ☐ Inject di Composable
- ☐ Display info



Waktu: 20 menit

LATIHAN 2: KOIN DI SETUP

Setup Dependency Injection untuk Notes App

AppModule.kt

```
val appModule = module {  
    // Database  
    single { DatabaseDriverFactory() }  
    single { AppDatabase(get<DatabaseDriverFactory>()  
        .createDriver()) }  
  
    // Repositories  
    single { NoteRepository(get()) }  
    single { SettingsRepository(get()) }  
  
    // Platform services  
    single { DeviceInfo() }  
    single { NetworkMonitor() }  
  
    // ViewModels  
    viewModelOf(::NotesViewModel)  
    viewModelOf(::SettingsViewModel)  
}  
  
// TODO:  
// 1. Setup Koin initialization  
// 2. Inject with koinViewModel()
```

Checklist:

- ☐ Add Koin dependencies
- ☐ Create appModule
- ☐ Platform-specific module
- ☐ Initialize Koin
- ☐ Migrate existing code
- ☐ Use koinViewModel()
- ☐ Test DI graph



Waktu: 25 menit

LATIHAN 3: NETWORK STATUS INDICATOR

Tampilkan status koneksi internet

NetworkIndicator.kt

```
@Composable
fun NetworkStatusIndicator() {
    val networkMonitor: NetworkMonitor = koinInject()
    val isConnected by networkMonitor
        .observeConnectivity()
        .collectAsState(initial = true)

    AnimatedVisibility(visible = !isConnected) {
        Surface(
            modifier = Modifier.fillMaxWidth(),
            color = MaterialTheme.colors.error
        ) {
            Row(modifier = Modifier.padding(8.dp)) {
                Icon(Icons.Default.CloudOff, "Offline")
                Spacer(Modifier.width(8.dp))
                Text("No Internet Connection")
            }
        }
    }
}
```

Checklist:

- ☐ NetworkMonitor expect
- ☐ Android actual
- ☐ iOS actual (stub ok)
- ☐ Register di Koin
- ☐ Create composable
- ☐ Add animation
- ☐ Test airplane mode



Waktu: 20 menit

BEST PRACTICES

✓ Keep expect minimal

Hanya deklarasi, logic di common code

✓ Use interfaces

Abstract dengan interface, inject implementation

✓ Single module pattern

Satu Koin module per layer/feature

✓ Lazy injection

Gunakan lazy untuk heavy dependencies

✓ Test DI graph

checkModules() untuk verify dependencies

✓ Document platform code

Jelaskan mengapa berbeda per platform

COMMON PITFALLS

✗ Missing actual implementation

✓ Compiler error - pastikan semua target punya actual

✗ Context leaks di Android

✓ Gunakan Application context, bukan Activity

✗ Circular dependency

✓ Restructure modules, gunakan lazy injection

✗ Platform code di common

✓ Gunakan expect/actual atau interface

RANGKUMAN PERTEMUAN 8

expect/actual

Deklarasi di common, implementasi per platform

Koin DI

Lightweight DI dengan Kotlin DSL

Platform APIs

Device info, battery, network, dll

Permissions

Runtime permission handling

Koin Module

single{}, factory{}, viewModelOf()

Architecture

Interface di common, implementation di platform



Platform-specific code yang clean = KMP yang powerful!

TUGAS PRAKTIKUM MINGGU 8

Bobot: 4% | Deadline: Sebelum Pertemuan 9

Deskripsi Tugas:

Upgrade Notes App dengan Platform Features:

1. Setup Koin Dependency Injection untuk seluruh app
2. Implementasi DeviceInfo dengan expect/actual
3. Implementasi NetworkMonitor dengan expect/actual
4. Tampilkan Device Info di Settings screen
5. Tampilkan Network Status indicator di main screen
6. Semua dependencies di-inject melalui Koin

Format Pengumpulan:

- Push ke GitHub repository (branch: week-8)
- README: Architecture diagram, screenshot device info & network indicator
- Video demo (45 detik) menunjukkan DI, device info, network status (on/off)

RUBRIK PENILAIAN TUGAS

Komponen	Bobot	Kriteria
Koin DI Setup	25%	All dependencies properly injected
expect/actual Pattern	25%	DeviceInfo & NetworkMonitor implemented
UI Integration	20%	Device info & network indicator displayed
Architecture	20%	Clean separation, proper modules
Code Quality	10%	Clean code, documentation
 Bonus (+10%) BatteryInfo expect/actual implementation  Penalti Terlambat: -10%/hari, Plagiat: 0		

SUMBER PUSTAKA

Dokumentasi

Kotlin Multiplatform

kotlinlang.org/docs/multiplatform.html

Dokumentasi

Koin

insert-koin.io/docs/quickstart/kotlin

Guide

expect/actual

kotlinlang.org/docs/multiplatform-connect-to-apis.html

Library

Accompanist Permissions

google.github.io/accompanist/permissions

Tutorial

KMP Platform APIs

kotlinlang.org/docs/multiplatform-mobile-integrate-in-existing-app.html

PREVIEW MINGGU DEPAN

Pertemuan 9: Testing dan Debugging



Unit Testing

kotlin.test, MockK, Turbine



UI Testing

Compose testing, screenshot tests



Debugging

Tools dan techniques



Code Coverage

Measuring test coverage



Persiapan:

- Selesaikan tugas Platform-Specific Features
- Baca dokumentasi kotlin.test
- Review testing best practices

KOIN CHEAT SHEET

Definition DSL

```
// Singleton - satu instance
single { Repository() }

// Factory - instance baru
factory { UseCase(get()) }

// ViewModel
viewModelOf(::MyViewModel)

// Named definition
single(named("api")) { "https://..." }

// Bind interface
single<Repository> { RepoImpl() }

// Parameter injection
factory { (id: Int) -> Detail(id) }
```

Injection

```
// In module - get dependency
single { Service(get()) }

// In class - inject
val repo: Repository by inject()

// In Composable
val vm = koinViewModel<MyViewModel>()
val svc: Service = koinInject()

// With parameters
val d = get<Detail> { parametersOf(123) }

// Named
val url: String = get(named("api"))

// Lazy
val repo: Repository by inject()
```

EXPECT/ACTUAL CHEAT SHEET

```
// ===== FUNCTION =====
// commonMain
expect fun getPlatform(): String

// androidMain
actual fun getPlatform(): String = "Android"

// iosMain
actual fun getPlatform(): String = "iOS"

// ===== CLASS =====
// commonMain
expect class FileManager {
    fun readFile(path: String): String
    fun writeFile(path: String, content: String)
}

// androidMain
actual class FileManager(private val context: Context) {
    actual fun readFile(path: String): String { /* Android impl */ }
    actual fun writeFile(path: String, content: String) { /* Android impl */ }
}

// ===== OBJECT =====
// commonMain
expect object Logger {
    fun log(message: String)
}
```



SESI TANYA JAWAB

Ada pertanyaan tentang Platform-Specific Features?

CHECKLIST SEBELUM PULANG

Pastikan Anda sudah:

- ☐ Memahami konsep expect/actual pattern
- ☐ Bisa membuat expect declaration dan actual implementation
- ☐ Mengerti manfaat Dependency Injection
- ☐ Bisa setup Koin module dan inisialisasi
- ☐ Mampu inject dependencies di ViewModel dan Composable
- ☐ Memahami cara akses platform-specific APIs
- ☐ Mengerti flow permission handling
- ☐ Mencatat deadline tugas Platform Features

Terima Kasih

Pertemuan 8: Platform-Specific Features

*"Write once, adapt everywhere.
That's the power of KMP."*

- Kotlin Multiplatform Philosophy

Selamat mengeksplorasi platform APIs! 🔧 📱